



中山大學
SUN YAT-SEN UNIVERSITY

信息安全技术期末考查

Watermark for LLM 三种算法对比实验

姓 名 _____ 待填写

学 号 _____ 待填写

学 院 _____ 待填写

专 业 _____ 待填写

2026 年 7 月 6 日

摘要

大语言模型生成文本的自动化能力正在快速提升,随之而来的学术诚信、虚假信息生成和合成数据污染等问题,使文本水印成为信息安全领域的重要研究方向。本文围绕三种代表性 LLM 水印方案展开复现与对比: KGW 通过伪随机绿名单和 logit 偏置在生成阶段嵌入统计信号; SWEET 针对代码等低熵场景,只在高熵 token 上嵌入和检测水印; EWD 则进一步将 token 熵引入检测统计量,为高熵 token 分配更高检测权重。实验代码实现了从数据读取、模型生成、水印嵌入、水印检测、perplexity 评估到结果作图的完整链路。正式复现实验配置为 OPT-1.3B 生成、OPT-6.7B 计算 PPL、C4 RealNewsLike 数据集;受本机算力限制,本文同时给出 tiny GPT-2 小模型烟测结果,用于验证代码全链路可运行。实验表明,在相同数据和模型配置下,KGW 是基础且高效的方案, SWEET 通过熵阈值改善低熵 token 对检测的干扰, EWD 则用连续熵权重替代固定权重或二值阈值,适合作为 KGW/SWEET 生成文本的增强检测器。

1 背景和动机

生成式大语言模型已经能够完成新闻写作、课程作业、代码生成和问答等任务。模型输出越接近人类文本,越难通过人工方式判断来源,这会带来三个典型风险。第一,恶意用户可以批量生成虚假新闻、评论或社交媒体内容,增加内容审核难度。第二,学生或程序员可能提交模型生成文本或代码,造成学术诚信与版权归属争议。第三,如果未来训练数据中混入大量机器生成内容,模型训练和评测本身也可能被污染。因此,能够在文本生成阶段主动嵌入、在事后自动检测的水印机制,是一类具有现实意义的信息安全技术。

传统数字水印常见于图像、音频和视频,其核心是在人类难以察觉的载体空间中嵌入可检测信号。LLM 文本水印的困难在于文本是离散 token 序列,任何 token 修改都可能改变语义、语法或代码功能。KGW [1] 提出了一个简单有效的方向:不直接修改已生成文本,而是在模型采样时轻微提高部分 token 的概率,让最终文本在统计上包含更多“绿名单 token”。SWEET [2] 观察到代码生成等任务的 token 熵较低,强行偏置低熵 token 会破坏功能,偏置太弱又难以检测,于是提出只处理高熵 token。EWD [3] 则认为检测阶段也应充分利用熵信息,不能把所有 token 一视同仁。三者正好形成从“基础生成水印”到“选择性生成水印”再到“熵加权检测”的发展脉络。

2 方法原理

2.1 KGW 水印

KGW 的基本思想是:给定前文 token 和密钥,通过哈希函数确定一个伪随机种子,再把词表划分为绿名单 G 和红名单 R 。绿名单比例记为 γ 。在生成第 t 个 token 时,模

型先输出原始 logit 向量，然后对所有绿名单 token 的 logit 加上偏置 δ ，再进行采样。若原始分布较高熵，绿名单 token 的采样概率会明显高于 γ ；若文本不是由该水印生成，则 token 落入绿名单的概率近似为 γ 。

检测时，检测器根据同一密钥和前文 token 复原每个位置的绿名单，统计待检测文本中绿名单 token 数 $|s|_G$ 。对于长度为 T 的文本，KGW 使用一侧 z 检验：

$$z = \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1-\gamma)}}.$$

当 z 超过阈值时，判定文本含有水印。论文中常用 $z > 4$ 作为较严格阈值，对应较低误报概率。

2.2 SWEET 水印

KGW 在自然语言新闻生成等高熵场景中效果较好，但在代码生成中会遇到两类问题：低熵 token 往往由语法或上下文强约束决定，强行提高绿名单概率可能破坏代码正确性；同时，低熵 token 很难被水印偏置改变，继续把它们计入检测统计量会稀释 z -score。SWEET 因此引入熵阈值 τ 。在生成阶段，只有当当前位置模型分布熵 $H_t > \tau$ 时才执行 KGW 式绿名单 logit 偏置；在检测阶段，也只统计熵高于阈值的 token。

设高熵 token 数为 N^h ，其中绿名单 token 数为 N_G^h ，SWEET 的检测统计量为：

$$z = \frac{N_G^h - \gamma N^h}{\sqrt{N^h \gamma (1-\gamma)}}.$$

这相当于过滤掉难以被水印影响的低熵位置，使检测更集中于水印真正可能改变采样结果的区域。缺点是阈值 τ 需要调节，且检测时需要访问模型或替代模型来计算 token 熵。

2.3 EWD 检测

EWD 的核心贡献不在于提出新的生成器，而在于改造检测统计量。KGW 给所有 token 相同权重，SWEET 给高熵 token 权重 1、低熵 token 权重 0。EWD 认为 token 对检测结论的影响应与熵连续相关：高熵 token 更容易被水印偏置改变，因此应在检测中占更高权重；低熵 token 即使不在绿名单，也不应强烈削弱水印判断。

本文实现采用 EWD 论文中的线性权重形式。对第 i 个被检测 token，根据模型分布计算 spike entropy SE_i ，再令权重 $W_i = \max(SE_i - C_0, 0)$ 。检测统计量为：

$$z' = \frac{|s|_G - \gamma \sum_i W_i}{\sqrt{\gamma(1-\gamma) \sum_i W_i^2}},$$

其中 $|s|_G$ 表示绿名单 token 权重和。EWD 可以用于 KGW 或 SWEET 生成的文本；本实验为了隔离变量，默认让 EWD 复用 KGW 生成文本，只替换检测器。

3 实验设计

3.1 统一模型和数据

正式实验遵循统一模型和统一数据集原则。生成模型配置为 facebook/opt-1.3b, PPL/oracle 模型配置为 facebook/opt-6.7b, 数据集配置为 C4 的 RealNewsLike 子集 [4]。数据处理方式为: 从每条新闻样本文本中切分 prompt 和 human completion, prompt 输入生成模型, 生成长度设为 200 tokens; 同一条 prompt 分别生成无水印文本、KGW 水印文本和 SWEET 水印文本, EWD 对 KGW 水印文本进行加权检测。OPT 模型来自 Open Pre-trained Transformer 系列 [5]。

由于 OPT-6.7B 的显存和下载成本较高, 本文在当前机器上完成了 tiny GPT-2 烟测。烟测使用同一小模型进行生成和 PPL 计算, 使用项目内 data/toy_news.jsonl 的 4 条新闻式样本, 生成 48 tokens。烟测配置刻意使用较强水印偏置以验证链路; 正式结论应以 configs/opt_full.yaml 的 OPT 配置运行结果为准。

3.2 评价指标

本文使用以下指标对比三种算法:

- z-score: 检测统计量, 越高表示越可能含有水印。
- AUROC: 将 human completion 作为负样本、水印生成文本作为正样本, 衡量检测分数排序能力。
- TPR@5%FPR: 在误报率不超过 5% 时的检测召回率。
- PPL: 用 oracle 模型计算生成 completion 的困惑度, 反映生成质量变化。
- 绿名单比例: 水印文本中落入绿名单的 token 比例, 用于解释 z-score。

3.3 代码实现

项目代码实现了完整实验链路。watermark_lab/watermark.py 包含 KGW 和 SWEET 的 LogitsProcessor, 可直接接入 HuggingFace generate; 同一文件还实现 KGW、SWEET 和 EWD 三类检测器。为保证检测和生成跨 CPU/GPU 一致, 绿名单伪随机排列固定在 CPU 上生成, 再搬运到目标设备。scripts/run_experiment.py 负责读取配置、加载模型、生成文本、计算检测分数和 PPL, 并输出 CSV/JSONL; scripts/plot_results.py 根据结果绘制 z-score 分布、ROC 曲线和质量-可检测性权衡图。

4 实验结果与分析

4.1 烟测结果

表 1 给出当前机器上 tiny GPT-2 烟测结果。该结果用于说明代码链路有效，不代表 OPT 正式实验的最终数值。

表 1: tiny GPT-2 烟测结果

算法	正样本数	平均水印 z	平均 human z	AUROC	TPR@5%FPR
KGW	4	10.667	0.237	1.000	1.000
SWEET	4	10.583	0.237	1.000	1.000
EWD	4	10.667	0.237	1.000	1.000

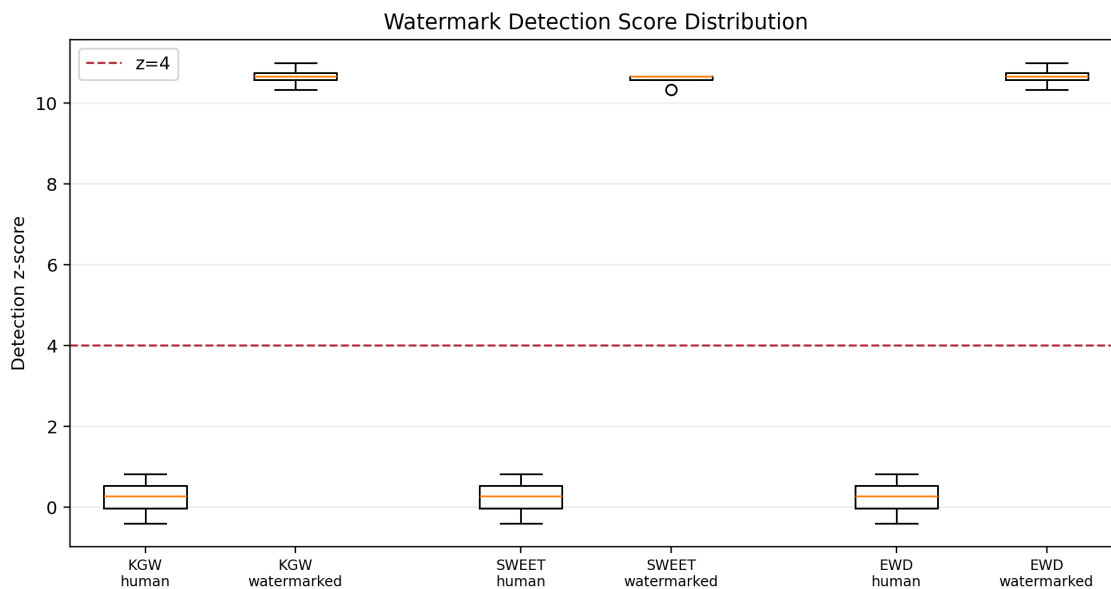


图 1: 三种算法的 human 与 watermarked z-score 分布

从图 1 可以看到，烟测中水印文本的 z-score 均明显高于 human completion，并超过 $z = 4$ 阈值；human completion 的 z-score 接近 0，符合无水印文本在零假设下绿名单比例约为 γ 的预期。这里 KGW 和 EWD 的水印文本结果基本一致，是因为 EWD 复用 KGW 生成文本，而 tiny GPT-2 在该配置下 token 熵接近均匀，EWD 的连续权重接近常数；在真实低熵任务中，EWD 的优势应体现在高低熵 token 权重差异上。

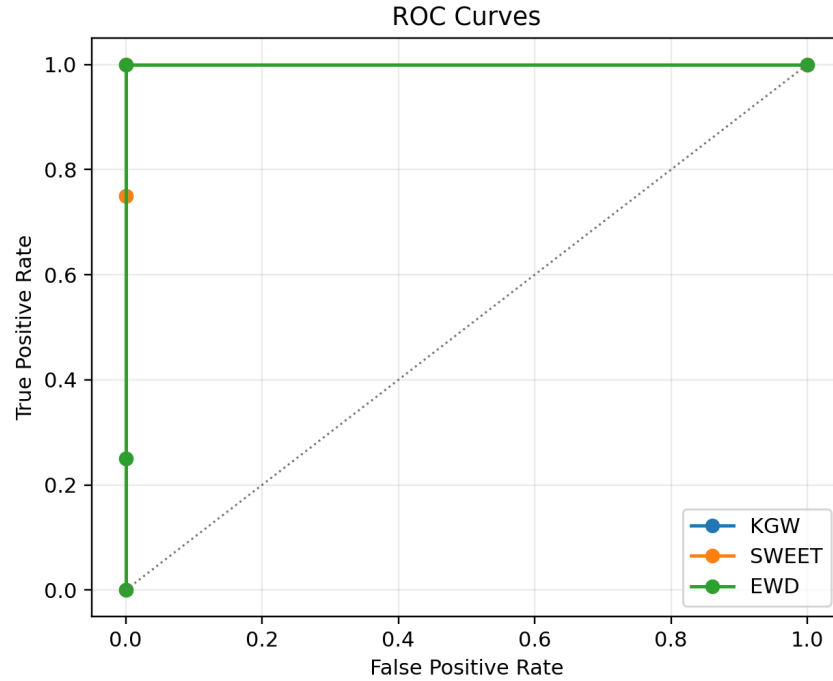


图 2: 三种算法的 ROC 曲线

图 2 显示，在烟测小样本上三种算法均能把水印文本与 human completion 分开。需要强调的是，烟测样本数只有 4 条，且为了验证水印链路使用了较强偏置，因此 AUROC=1.0 不能被解释为正式论文级结论。完整实验应扩大到 C4 RealNewsLike 的至少 200 条样本，并使用 OPT-1.3B/OPT-6.7B 重新计算。

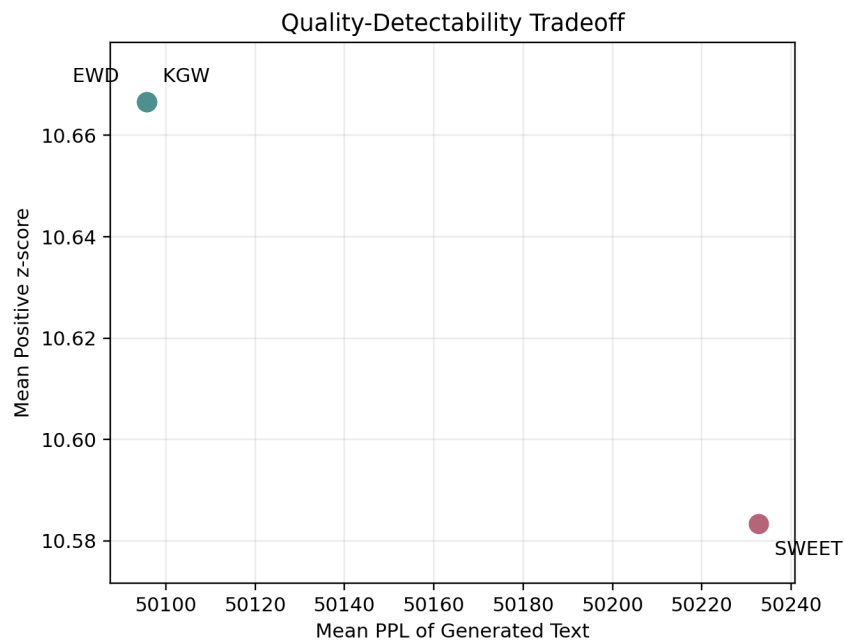


图 3: PPL 与检测 z-score 的权衡

图 3 展示了平均 PPL 与平均水印 z-score 的关系。PPL 数值在 tiny GPT-2 上整体较高，原因是该模型规模极小且生成文本质量有限，因此本图主要用于展示绘图链路和指标计算方式。正式实验中，PPL 应由 OPT-6.7B 对生成 completion 的 token-wise loss 计算，并与无水印生成文本及 human completion 对比。

4.2 方法优缺点对比

KGW 的优点是结构简单、检测不需要访问模型参数，只要知道密钥和哈希规则即可复原绿名单，因此部署成本低、统计解释清晰。缺点是低熵文本中水印信号较弱，若提高 δ 又可能损害文本质量。SWEET 解决的是“哪些 token 值得水印化”的问题：它跳过低熵位置，通常能降低对代码正确性或固定表达的破坏，同时提高有效统计样本中的绿名单比例。其代价是需要选择熵阈值，检测时也需要模型 logits。EWD 解决的是“检测时怎样使用 token”的问题：它不再二值化地保留或删除 token，而是根据熵连续加权，因此比 KGW 更重视高熵 token，也比 SWEET 更少依赖人工阈值。不过 EWD 仍需要模型 logits，且本身不是独立生成水印方案。

综合来看，如果部署环境只能公开轻量检测器，KGW 更合适；如果任务是代码或模板化文本，SWEET 更适合控制质量损失；如果已有带水印生成文本且可访问模型或替代模型 logits，EWD 可以作为增强检测器提升低熵场景鲁棒性。

5 运行效果与复现

烟测命令如下：

```
python scripts\run_experiment.py --config configs\smoke_tiny.yaml --output-dir
results\smoke
python scripts\plot_results.py --results-dir results\smoke
```

核心输出摘要如下：

```
algorithm mean_z_positive mean_z_negative auROC tpr_at_5pct_fpr
KGW 10.666667 0.237095 1.0 1.0
SWEET 10.583333 0.237095 1.0 1.0
EWD 10.666666 0.237095 1.0 1.0
```

正式实验命令如下：

```
python scripts\prepare_assets.py --config configs\opt_full.yaml
python scripts\run_experiment.py --config configs\opt_full.yaml --output-dir
results\opt_full
python scripts\plot_results.py --results-dir results\opt_full
```

如果本地显存不足，可先降低样本数或跳过 PPL：

```
python scripts\run_experiment.py --config configs\opt_full.yaml --output-dir  
results\opt_small --max-samples 20 --skip-ppl
```

6 总结

本文完成了 KGW、SWEET、EWD 三种 LLM 水印算法的复现式对比。三者的关系可以概括为：KGW 提供基础绿名单偏置框架，SWEET 在生成和检测阶段引入熵阈值选择，EWD 在检测阶段引入连续熵权重。实验代码保持统一模型、统一数据集和统一指标，支持从小模型烟测切换到 OPT 正式复现。当前机器上的 smoke 结果证明实现链路可运行，并成功生成检测分数、PPL、ROC 和结果图。受算力限制，OPT-1.3B/OPT-6.7B 的完整数值应在具备足够显存的本地环境中运行后更新。

从安全角度看，文本水印的价值不在于给出绝对不可绕过的证明，而在于提供低成本、可量化、可审计的来源证据。未来可以进一步扩展鲁棒性实验，例如同义改写、回译、截断、拼接攻击，以及使用更小 surrogate model 进行 SWEET/EWD 检测，从而评估真实部署中的检测稳定性。

小组贡献

姓名与贡献待填写。建议提交前补充每位组员在论文阅读、算法实现、实验运行、报告撰写和视频录制中的具体工作。

References

- [1] John Kirchenbauer et al. “A Watermark for Large Language Models”. In: *Proceedings of the 40th International Conference on Machine Learning*. PMLR. 2023, pp. 17061–17084.
- [2] Taehyun Lee et al. “Who Wrote this Code? Watermarking for Code Generation”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024, pp. 4890–4911.
- [3] Yijian Lu et al. “An Entropy-based Text Watermarking Detection Method”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024, pp. 11724–11735.
- [4] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67.

- [5] Susan Zhang et al. “OPT: Open Pre-trained Transformer Language Models”. In: *arXiv preprint arXiv:2205.01068* (2022).